Tellix — User Guide

Tellix is a lightweight Windows screen recorder that captures your screen plus your microphone (and optionally Google Meet / Zoom / any system audio), produces a clean MP4 with captions embedded, and writes a timestamped transcript. Everything runs locally on your machine — no internet calls beyond a one-time Whisper model download.

This guide walks through how to use the app end-to-end.

Quick start

Launch Tellix and the main window opens. Three controls matter before you start recording.

The **Microphone** dropdown selects your input device. The first entry is usually correct; change it only if you have a USB mic or headset you want to favour. The **Model** dropdown picks the Whisper model size used for transcription. `base` is the default and balances speed against accuracy. Larger sizes (`small`, `medium`) are more accurate but slower and use more memory. The **Include system audio** checkbox controls whether Tellix also captures whatever is playing through your speakers or headphones — tick this when recording a meeting or anything where you want the other side captured. Leave it unchecked for screen-only or solo narration. If the checkbox is greyed out with a red "WASAPI loopback unavailable" note, run `pip install soundcard` in your venv and relaunch.

Click **Start recording**, work as normal (you don't need to keep the app window focused), and click **Stop** when you're done.

What happens after Stop

Tellix runs through three steps automatically. First it closes the screen and audio captures. Then it runs the full-quality Whisper pass on the recorded audio — this is the slow step, usually a quarter of your recording's length on CPU, much faster on an NVIDIA GPU; the status bar shows "Transcribing (final pass)..." while this happens. Finally it muxes the video, audio, and transcript into a single `recording.mp4` with embedded subtitles, then offers to open the output folder.

Where your recording lives

Sessions land in `output/tellix-<date>-<time>/` next to the app. Each session folder contains:

File	Contents
`recording.mp4`	Final video with audio and a soft-subtitle track. This is what you share.
`recording.srt`	Standalone subtitle file. Identical content to the embedded track.
`recording.txt`	Plain transcript without timestamps. Good for copy-paste.

File	Contents
`screen.mp4`	Raw video-only intermediate, kept for debugging.
`mic.wav`	Microphone capture at 16 kHz mono.
`system.wav`	Loopback capture, only present when system audio was enabled.
`mixed.wav`	Mic + system audio combined, only present when system audio was on.

If you want to clean up disk space, the three "raw" files (`screen.mp4`, `mic.wav`, `system.wav`, `mixed.wav`) can all be deleted safely — `recording.mp4` is fully self-contained.

Viewing the captions

The captions are embedded inside `recording.mp4` as a soft subtitle stream. Most players don't display them by default; you toggle them on the same way you toggle CC on YouTube. Quick reference:

Player	How to enable captions
VLC	Subtitle menu → Sub Track → "English"
Windows Media Player Legacy	Play menu → Captions and Subtitles → On
Movies & TV (Windows 11)	Click the CC icon at the bottom-right
Chrome / Edge / Firefox	Drag the file into a tab, click the CC button
Premiere / DaVinci	Auto-detected as an embedded subtitle stream

If you'd rather have captions burned permanently into the video pixels (always visible without needing to toggle them), this is a planned enhancement — see the Roadmap section in RELEASE.md.

Recovering a transcript

If a session ends up with an empty or unhelpful transcript, you can re-run the transcription on the saved audio without re-recording. Open a terminal in the Tellix folder and run:

```
python retranscribe.py
```

With no arguments it auto-picks the most recent session under `output/`. To target a specific session or WAV file directly, give it the path:

```
python retranscribe.py output\tellix-20260513-140745
python retranscribe.py path\to\mic.wav --model medium
python retranscribe.py output\tellix-20260513-140745 --language en
```

It overwrites `recording.srt` and `recording.txt` in that session folder. The `--model` flag accepts `tiny`, `base`, `small`, or `medium`. The `--language` flag accepts any Whisper language code (e.g. `en`, `es`, `de`); omit it to auto-detect.

Troubleshooting

"FFmpeg not found" comes up if you're running from source and `bin\ffmpeg.exe` isn't where Tellix expects it. Drop a static Windows build of `ffmpeg.exe` into the `bin\` folder, or add ffmpeg to your PATH. This doesn't apply to the bundled `Tellix.exe` — FFmpeg is included inside.

"Audio capture failed" with a PortAudio MME error means your selected mic didn't accept the requested format through Windows' legacy audio API. Tellix tries up to seven fallback combinations automatically. If you still hit this after a rebuild, the most likely causes are a corporate mic-access policy, the mic being held exclusively by another app (e.g., Zoom or Teams with the mic muted), or a driver problem. Close other audio apps and try again.

"WASAPI loopback unavailable" in red next to the system-audio checkbox means the `soundcard` library isn't installed in your venv. Run `pip install soundcard` and relaunch. The bundled `.exe` always includes it.

`recording.txt` **says "(no speech detected — check mic.wav levels or speak louder)"** means Whisper found nothing it considered speech. Open `mic.wav` in any media player to verify your voice is actually present at a normal volume. Common causes are mic input level set too low in Windows Sound settings, the mic being muted, or the wrong mic being selected.

Captions don't appear in `recording.mp4` when you play it on another machine — they're embedded but most players hide them by default. See the "Viewing the captions" section above.

"Windows protected your PC" SmartScreen dialog when launching `Tellix.exe` on a new machine — that's Microsoft Defender being cautious about unsigned executables. Click **More info** → **Run anyway** to proceed. After the first launch, Windows trusts that .exe on that machine and won't ask again. To skip even that one prompt for users you distribute to, code-sign the binary with an EV certificate.

First launch of the .exe takes 10+ seconds — normal for PyInstaller bundled executables. The bootloader extracts everything to a temp directory on first run; subsequent launches reuse the cache and are quick.

First transcription is slow — Whisper downloads the model (~150 MB for `base`, ~500 MB for `small`) from HuggingFace on first use. It's cached in `%USERPROFILE%\.cache\huggingface\` and reused forever after.

System requirements

Tellix runs on Windows 10 or 11. It needs about 500 MB of free disk for the Whisper model cache, 4 GB of RAM for the `base` model, and 8 GB recommended if you want to use `small` or `medium`. Any microphone supported by the Windows audio stack works — Tellix auto-negotiates the format. An NVIDIA GPU is optional and gives roughly a 5× speedup on transcription via CUDA; the app falls back to CPU automatically when no GPU is available.

Privacy

Everything runs on your machine. Audio never leaves your computer. The Whisper model is downloaded once from HuggingFace on first transcription and cached locally; subsequent transcriptions run fully offline. No telemetry, no analytics, no cloud uploads.